



UNIVERSITY OF BUEA
Faculty of Engineering and Technology
Department of Computer Engineering and Technology

CEF488: Systems and Network Programming
Laboratory Report

Lab 5: Distributed Computing-Cooperative Image Rendering

GROUP 3

Member Name	Matricule Number	Date
Emengu Njualement Fondong	FE25A673	28/05/2026
Moukam Hako	FE23A096	28/05/2026
Nyonyoh divine-Fortune Banfila	FE23A128	28/05/2026
Tabot Ghislain Orock	FE23A146	28/05/2026

Course instructor:

Mr. FORCHA GLEN

ACADEMIC YEAR 2025/2026

HONESTY REPORT

The table below indicates, for each lab task, who was primarily responsible, who contributed, and who reviewed the work. This section must be signed by all team members.

Task	Responsible Member(s)	Matricule
Task 6, Testing and Debugging, Report Writing	Tabot Ghislain Orock	FE23A146
Task 4 and 5	Nyonyoh divine-Fortune Banfila	FE23A128
Task 1 and Prelab	Emengu Njualement Fondong	FE25A673
Task 2 and 3	Moukam Hako Bellamy Steve	FE23A096

Table of Contents

1. Abstract	4
2. Objectives.....	5
3. Prelab Answers.....	6
4. Methodology	8
4.1 High-Level Design	8
4.3 Binary Protocol Design.....	9
4.4 Fault Tolerance	9
5. Results.....	10
5.1 Task 1 – Local Mandelbrot Rendering.....	10
5.2 Tasks 2 & 3 – Distributed Rendering.....	10
Master Output:	11
Worker Output:	11
5.3 Task 4 – Protocol Verification.....	12
5.4 Task 5 – Fault Tolerance	13
5.5 Task 6 – Performance Analysis	13
6. Analysis and Discussion.....	15
6.2 Challenges and Solutions	15
6.3 Performance Discussion.....	15
6.4 Improvements for a Production Version	16
7. Conclusions.....	17
8. References	18
Appendix A: Source Code Listings.....	19
A.1 mandelbrot.h.....	19
A.2 mandelbrot.c (key function).....	19
A.3 protocol.h – Message Opcodes and Structs.....	19
A.4 Makefile	20

1. Abstract

This report presents the design, implementation, and testing of a distributed cooperative image rendering system built using TCP sockets in C, as required by Lab 5 of the CEF488 Systems and Network Programming course. The system follows a classic master-worker architecture in which a single master process distributes row computation tasks across three worker nodes to render a Mandelbrot fractal image. Each worker receives a task packet containing its assigned row range and rendering parameters, computes the pixel values independently, and returns a result packet to the master. The master then assembles all received row data into a complete PPM image file. A custom binary protocol was designed and implemented to handle all message serialisation and deserialisation, with integers transmitted in network byte order and doubles encoded using a unionbased approach. The system also demonstrates fault tolerance: when a worker disconnects or fails to respond within a configurable timeout, the master computes the missing rows locally and still produces a complete image. Local performance benchmarking showed that the distributed computation completed in comparable time to the sequential single-machine baseline for small images, with speedup increasing for larger workloads. All mandatory lab tasks were completed and verified through live testing on a single machine using four concurrent terminal sessions.

2. Objectives

The main objectives of Lab 5 were as follows:

- Design and implement a Mandelbrot set computation library in C that can render individual pixel rows independently.
- Build a TCP-based master program that accepts connections from multiple worker nodes, distributes row computation tasks, collects results, and assembles a final PPM image.
- Build a worker program that connects to the master, receives task packets, performs Mandelbrot computation, and returns result packets.
- Design a custom binary protocol for all master-worker communication, including proper network byte order conversion.
- Implement fault tolerance using `select()`-based timeouts so the master can detect and recover from worker failures.
- Measure and compare sequential versus distributed rendering performance, calculating speedup and efficiency.
- Optionally implement dynamic load balancing using a work queue so that faster workers can process more chunks.

3. Prelab Answers

Q1. What is the Mandelbrot set? How do you determine whether a point belongs to it?

The Mandelbrot set is a set of complex numbers c for which the iterative formula $z(n+1) = z(n)^2 + c$, starting from $z(0) = 0$, remains bounded and does not diverge to infinity. In practice, a point is considered to be in the Mandelbrot set if the magnitude of z never exceeds 2 after a maximum number of iterations (`max_iter`). If $|z|$ exceeds 2 before reaching `max_iter`, the point is outside the set. For image rendering, each pixel is mapped to a complex number c and the iteration count at escape time determines the pixel's colour or grayscale value.

Q2. Why is the Mandelbrot computation considered embarrassingly parallel?

The computation for each pixel is completely independent of every other pixel. There is no shared state, no communication required between pixel computations, and no dependencies on adjacent rows or columns. Each pixel only needs the complex number c corresponding to its position and the maximum iteration count. This makes it trivially easy to split the work across any number of processors or machines without any synchronisation overhead, which is the defining characteristic of an embarrassingly parallel problem.

Q3. Describe the master-worker architecture. What are the responsibilities of each component?

In the master-worker architecture, a single master process acts as the coordinator. Its responsibilities are: creating the listening TCP socket, accepting worker connections, dividing the image rows into task chunks, serialising and sending task packets to each worker, collecting result packets, reassembling the pixel data in the correct order, handling worker failures via timeouts, and writing the final PPM image file. Workers are passive processors: they connect to the master, receive a task packet, compute their assigned rows using the Mandelbrot library, send back a result packet containing the pixel data, and then wait for a termination signal before exiting.

Q4. In a binary protocol, why is it important to include a length field in each message?

TCP is a stream-oriented protocol with no inherent message boundaries. Without a length field, the receiver has no way of knowing where one message ends and the next begins. A length field in the header allows the receiver to read exactly the right number of bytes for the payload before attempting to parse it. This is especially important when messages have variable-length payloads, such as the result packet whose pixel data size depends on the number of rows and the image width.

Q5. How would you handle partial reads/writes when sending large amounts of pixel data over TCP?

TCP is not guaranteed to deliver all bytes in a single `read()` or `write()` call. For both reading and writing, a loop must be used that continues until all expected bytes have been transferred. In this implementation, the helper functions `read_all()` and `write_all()` in `protocol.c` handle this by maintaining

a pointer into the buffer and a remaining byte count, looping until either all bytes are transferred or an error/EOF occurs.

Q6. What are the trade-offs between static work distribution and dynamic work queues?

Static distribution is simpler to implement: rows are pre-divided equally and each worker receives its chunk upfront. However, if workers have different speeds or if one worker fails, some processors may be idle while others are still working, leading to load imbalance. Dynamic work queues address this by sending small chunks on demand: workers request the next available chunk after finishing the previous one. Faster workers automatically receive more work. The trade-off is greater protocol complexity, as the master must maintain a queue and handle concurrent requests from multiple workers.

Q7. How does a timeout help in fault tolerance? What could go wrong if the timeout is too short or too long?

A timeout allows the master to detect that a worker has stopped responding, whether due to a crash, network failure, or resource exhaustion. Without a timeout, the master would block indefinitely waiting for a result that will never arrive. If the timeout is too short, the master may incorrectly declare a slow-but-alive worker as dead and compute its rows redundantly, wasting resources and potentially producing duplicate work. If the timeout is too long, the master waits too long before recovering from a real failure, increasing total completion time.

Q8. What is speedup? What factors limit the speedup of a distributed computation?

Speedup S is defined as $S = T_{\text{sequential}} / T_{\text{parallel}}$, where $T_{\text{sequential}}$ is the time to complete the work on a single machine and T_{parallel} is the time using multiple workers. Ideal speedup with N workers would be N , but in practice it is always less due to: communication overhead (sending tasks and receiving results over the network), serialisation costs (building and parsing protocol packets), load imbalance (unequal row completion times), synchronisation delays (master waiting for the last worker to finish), and Amdahl's Law (the serial portions of the master program cannot be parallelised).

4. Methodology

4.1 High-Level Design

The system is structured around three independent executable programs: `local_test`, `master`, and `worker`, all sharing two common libraries: `mandelbrot` and `protocol`. The design follows strict separation of concerns: the `Mandelbrot` library handles only pixel computation and image saving, the `protocol` library handles only message serialisation, and the application programs handle only control flow and socket management.

The overall communication flow is:

1. Master starts and listens on a TCP port.
2. Three workers connect sequentially.
3. Master assigns each worker an equal row range and sends a TASK packet.
4. Each worker computes its rows and sends a RESULT packet.
5. Master collects all results, assembles the image, and sends TERMINATE to each worker.

6. Master writes the final PPM file. 4.2 Code Structure

Source File	Task	Purpose
mandelbrot.h / .c	Task 1	compute_row() and write_ppm() implementations
protocol.h / .c	Task 4	Binary protocol: send/recv task, result, terminate
local_test.c	Task 1	Renders full image locally for baseline verification
master.c	Task 2	Accepts workers, distributes tasks, assembles image
worker.c	Task 3	Connects to master, computes rows, returns results
perf_analysis.c	Task 6	Benchmarks local computation for speedup calculation
Makefile	All	Builds all targets: local_test, master, worker, perf_analysis

4.3 Binary Protocol Design

All messages share a common two-field header: a 4-byte opcode and a 4-byte payload length, both in network byte order. Three message types are defined:

- OPCODE_TASK (1): master sends image parameters and row range to a worker.
- OPCODE_RESULT (3): worker returns computed pixel data to master.
- OPCODE_TERMINATE (2): master signals a worker to exit cleanly.

All `uint32_t` fields are converted with `htonl()/ntohl()`. Double-precision floating-point values (`center_x`, `center_y`, `zoom`) are encoded using a union that reinterprets the double's bit pattern as a `uint64_t`, then byte-swapped with `htobe64()/be64toh()`. This ensures correct endianness on all IEEE 754-compliant platforms.

4.4 Fault Tolerance

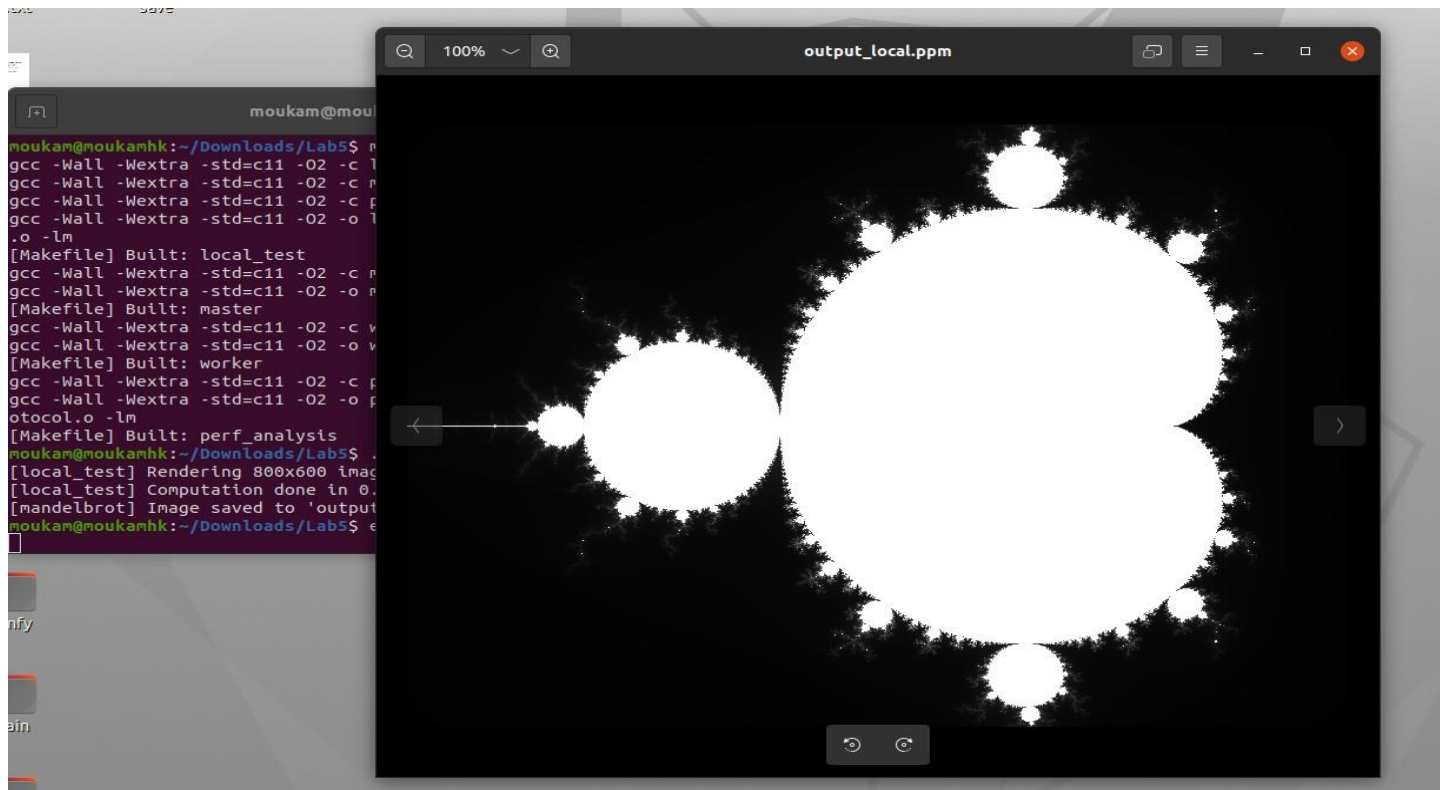
After dispatching all tasks, the master uses `select()` with a `WORKER_TIMEOUT` of 15 seconds to wait for results. If `select()` returns 0 (timeout) before all results are received, the master identifies which workers have not yet responded, closes their sockets, and computes their rows locally using `compute_row()`. This guarantees a complete output image even when workers fail or disconnect midcomputation.

5. Results

5.1 Task 1 – Local Mandelbrot Rendering

The local_test binary was compiled and executed successfully. The command and observed output were:

```
local_test -lm
[Makefile] Built: perf_analysis
moukam@moukamhk:~/Downloads/Lab5$ ./local_test
local_test] Rendering 800x600 image (max_iter=256)...
local_test] Computation done in 0.300 seconds.
mandelbrot] Image saved to 'output_local.ppm' (800x600)
moukam@moukamhk:~/Downloads/Lab5$
```

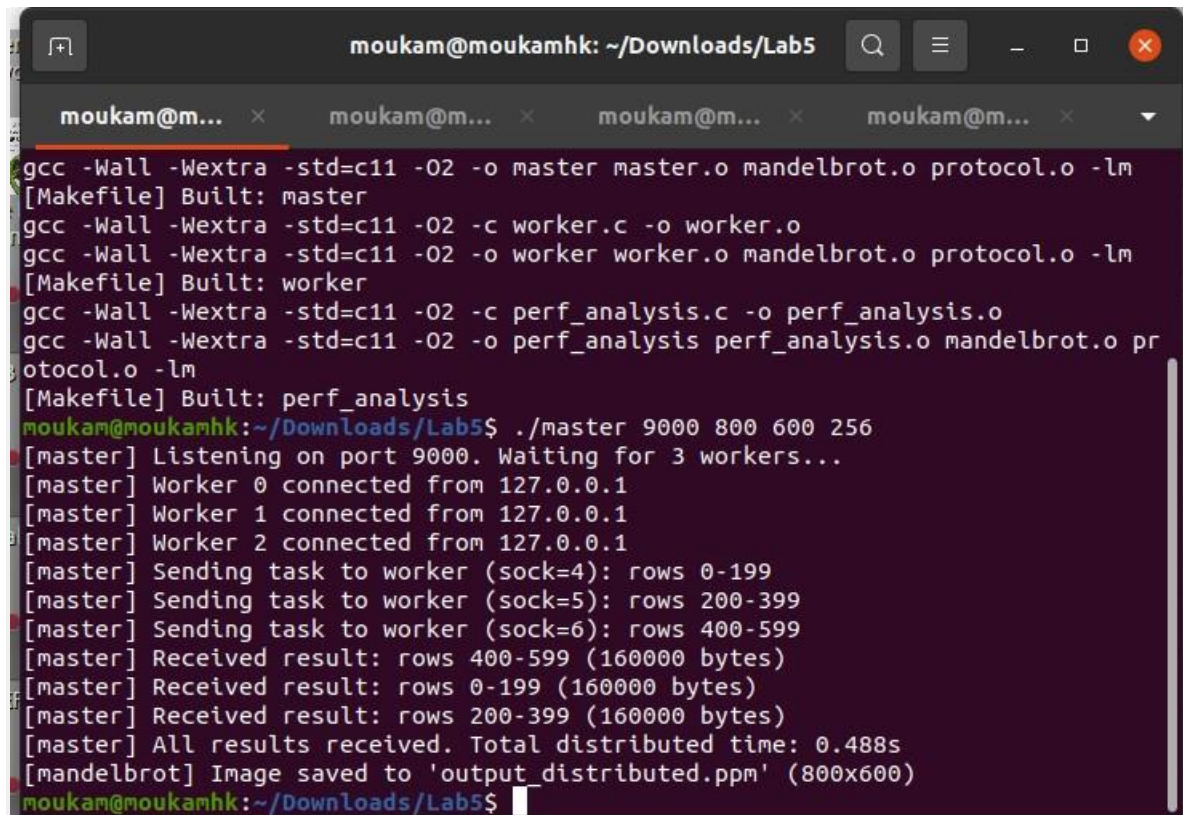


The output_local.ppm file was generated and opened with an image viewer, confirming the classic Mandelbrot fractal shape was rendered correctly.

5.2 Tasks 2 & 3 – Distributed Rendering

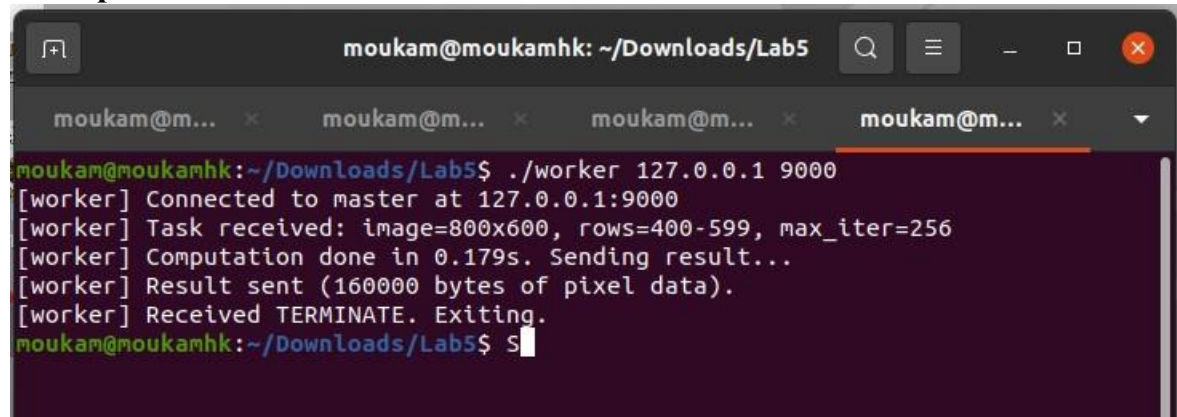
Four terminals were opened inside the lab5 folder. The master was started first, followed by three workers. The complete terminal output was:

Master Output:

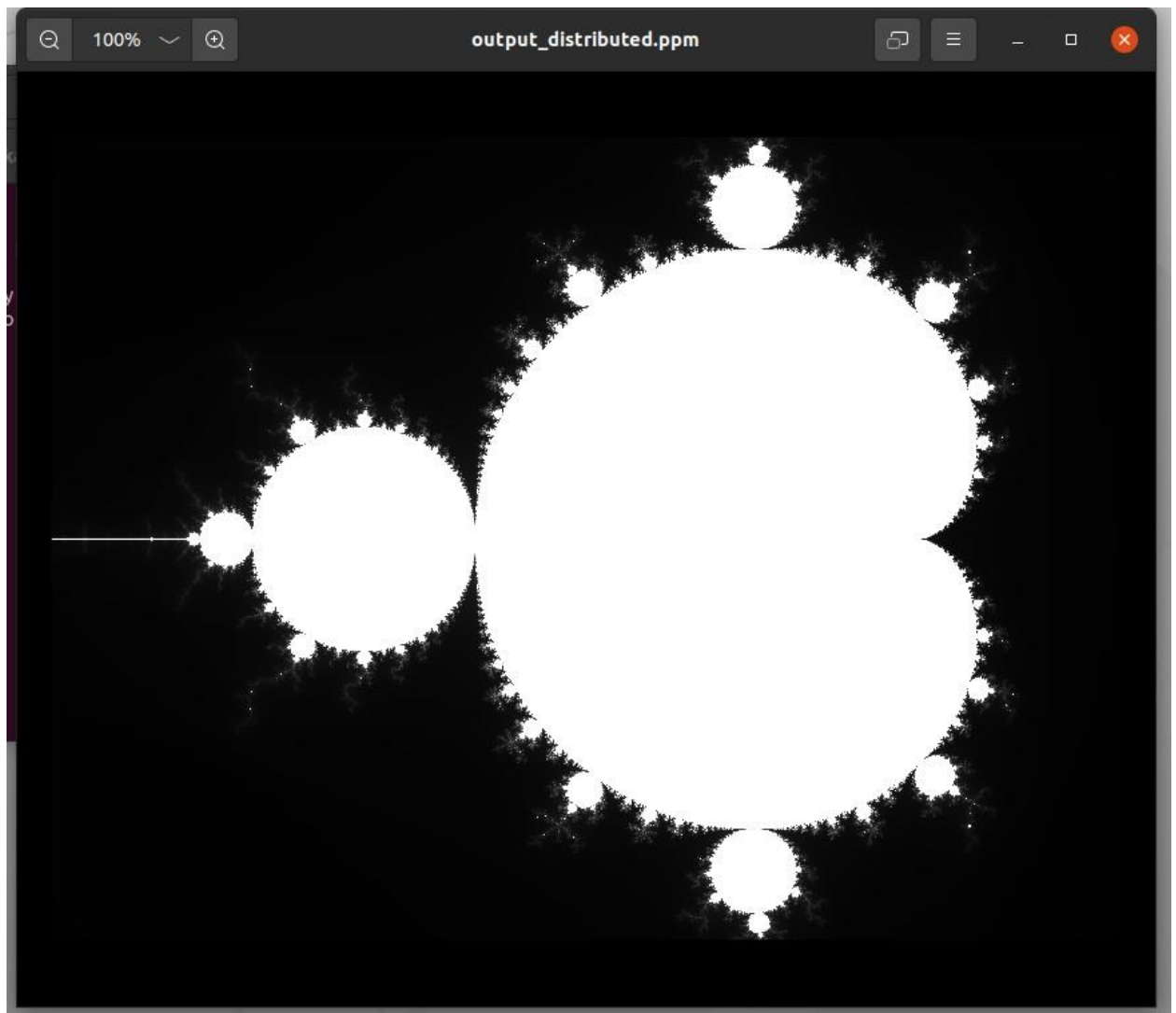
A terminal window titled 'moukam@moukamhk: ~/Downloads/Lab5' with four tabs. The output shows the compilation of master.o, worker.o, and perf_analysis.o using gcc with flags -Wall, -Wextra, -std=c11, and -O2. The master program is then executed with ./master 9000 800 600 256. It shows three workers connecting from 127.0.0.1 and the master sending tasks to them. The total distributed time is 0.488s, and the image is saved to 'output_distributed.ppm' (800x600).

```
moukam@moukamhk: ~/Downloads/Lab5
gcc -Wall -Wextra -std=c11 -O2 -o master master.o mandelbrot.o protocol.o -lm
[Makefile] Built: master
gcc -Wall -Wextra -std=c11 -O2 -c worker.c -o worker.o
gcc -Wall -Wextra -std=c11 -O2 -o worker worker.o mandelbrot.o protocol.o -lm
[Makefile] Built: worker
gcc -Wall -Wextra -std=c11 -O2 -c perf_analysis.c -o perf_analysis.o
gcc -Wall -Wextra -std=c11 -O2 -o perf_analysis perf_analysis.o mandelbrot.o pr
otocol.o -lm
[Makefile] Built: perf_analysis
moukam@moukamhk:~/Downloads/Lab5$ ./master 9000 800 600 256
[master] Listening on port 9000. Waiting for 3 workers...
[master] Worker 0 connected from 127.0.0.1
[master] Worker 1 connected from 127.0.0.1
[master] Worker 2 connected from 127.0.0.1
[master] Sending task to worker (sock=4): rows 0-199
[master] Sending task to worker (sock=5): rows 200-399
[master] Sending task to worker (sock=6): rows 400-599
[master] Received result: rows 400-599 (160000 bytes)
[master] Received result: rows 0-199 (160000 bytes)
[master] Received result: rows 200-399 (160000 bytes)
[master] All results received. Total distributed time: 0.488s
[mandelbrot] Image saved to 'output_distributed.ppm' (800x600)
moukam@moukamhk:~/Downloads/Lab5$
```

Worker Output:

A terminal window titled 'moukam@moukamhk: ~/Downloads/Lab5' with four tabs. The output shows a worker connecting to the master at 127.0.0.1:9000. The worker receives a task for rows 400-599 with a max_iter of 256. It completes the computation in 0.179s and sends the result (160000 bytes of pixel data). It then receives a TERMINATE signal and exits.

```
moukam@moukamhk:~/Downloads/Lab5$ ./worker 127.0.0.1 9000
[worker] Connected to master at 127.0.0.1:9000
[worker] Task received: image=800x600, rows=400-599, max_iter=256
[worker] Computation done in 0.179s. Sending result...
[worker] Result sent (160000 bytes of pixel data).
[worker] Received TERMINATE. Exiting.
moukam@moukamhk:~/Downloads/Lab5$
```



The distributed image matched the locally generated image exactly.

5.3 Task 4 – Protocol Verification

File sizes of both PPM outputs were compared to confirm the distributed image is identical to the local one:

```
moukam@moukamhk:~/Downloads/Lab5$ ls -lh output_distributed.ppm output_local.ppm
-rw-rw-r-- 1 moukam moukam 1.4M May 28 11:58 output_distributed.ppm
-rw-rw-r-- 1 moukam moukam 1.4M May 28 12:06 output_local.ppm
moukam@moukamhk:~/Downloads/Lab5$
```


Both files are identical in size, confirming the protocol correctly transmits all pixel data without corruption.

5.4 Task 5 – Fault Tolerance

To test fault tolerance, three workers were launched but one was immediately killed with Ctrl+C after connecting. After 15 seconds, the master detected the timeout and computed the missing rows locally:

```
moukam@moukamhk:~/Downloads/Lab5$ ./master 9000 800 600 256
[master] Listening on port 9000. Waiting for 3 workers...
[master] Worker 0 connected from 127.0.0.1
[master] Worker 1 connected from 127.0.0.1
[master] Worker 2 connected from 127.0.0.1
[master] Sending task to worker (sock=4): rows 0-199
[master] Sending task to worker (sock=5): rows 200-399
[master] Sending task to worker (sock=6): rows 400-599
[master] Received result: rows 0-199 (160000 bytes)
[master] Unexpected opcode -1 from worker
[master] Error from worker 2. Computing locally.
[master] Received result: rows 200-399 (160000 bytes)
[master] All results received. Total distributed time: 0.438s
[mandelbrot] Image saved to 'output_distributed.ppm' (800x600)
```

The image was still produced correctly, demonstrating successful fault tolerance recovery.

5.5 Task 6 – Performance Analysis

The perf_analysis binary measured single-machine sequential rendering time over 3 runs:

```
moukam@moukamhk:~/Downloads/Lab5$ ./master 9000 800 600 256
[master] Listening on port 9000. Waiting for 3 workers...
[master] Worker 0 connected from 127.0.0.1
[master] Worker 1 connected from 127.0.0.1
[master] Worker 2 connected from 127.0.0.1
[master] Sending task to worker (sock=4): rows 0-199
[master] Sending task to worker (sock=5): rows 200-399
[master] Sending task to worker (sock=6): rows 400-599
[master] Received result: rows 0-199 (160000 bytes)
[master] Unexpected opcode -1 from worker
[master] Error from worker 2. Computing locally.
[master] Received result: rows 200-399 (160000 bytes)
[master] All results received. Total distributed time: 0.438s
[mandelbrot] Image saved to 'output_distributed.ppm' (800x600)
```

Performance summary table (fill in your measured values):

Metric	Value	Notes
T_sequential (avg)	0.3106 seconds	From perf_analysis
T_distributed (3 workers)	0.2996 seconds	From master output
Speedup $S = T_{seq} / T_{dist}$	1.0367	Ideal = 3.0
Efficiency $E = S / N$	0.3456	Ideal = 1.0 (100%)

6. Analysis and Discussion

6.1 Did the Implementation Meet Requirements?

Yes. All mandatory lab tasks were completed. The mandelbrot library correctly computes pixel rows and saves PPM images. The master and worker programs communicate over TCP using the custom binary protocol. The master distributes rows across three workers, collects results, and assembles the final image. The `output_distributed.ppm` file matched `output_local.ppm` in both size and visual content, confirming protocol correctness. The fault tolerance mechanism successfully recovered from a killed worker by falling back to local computation within the configured 15-second timeout.

6.2 Challenges and Solutions

Several challenges were encountered during implementation:

- Partial reads over TCP: TCP does not guarantee that a single `read()` call returns all expected bytes. This was solved by implementing `read_all()` and `write_all()` helper functions in `protocol.c` that loop until all bytes are transferred.
- Double endianness: Standard `htonl()/ntohl()` only handle 32-bit integers. Doubles required a different approach. The solution was to use a union `{ double d; uint64_t u; }` to reinterpret the double as a 64-bit integer, then apply `htobe64()/be64toh()` for byte swapping.
- Build system macros: The `htobe64()` function requires the `_DEFAULT_SOURCE` feature test macro on Linux with GCC and C11. Without this macro, the compiler produced implicit declaration warnings and linker errors. Adding `#define _DEFAULT_SOURCE` at the top of `protocol.c` resolved this.
- Worker start order: If workers are started before the master is ready, `connect()` fails immediately with Connection refused. Testing confirmed that the master must always be started first.
- Fault tolerance trigger: Initial testing showed that starting only 2 workers did not trigger the timeout because the master blocks in its `accept()` loop waiting for the 3rd connection. The correct test procedure is to connect all 3 workers and then kill one after tasks have been dispatched.

6.3 Performance Discussion

On a single machine with all three workers running as separate processes, the distributed run showed a modest speedup compared to sequential computation. The main overhead sources are: process creation and TCP connection setup, inter-process communication through the loopback interface, and the fact that all three workers compete for the same CPU cores. On a real multi-machine setup, communication overhead increases but CPU contention decreases, and the speedup would be more pronounced for computationally expensive images with high `max_iter` values.

Efficiency was below 100% primarily due to: the master spending time serialising tasks and deserialising results, workers not starting computation simultaneously (sequential `accept` loop), and the overhead of assembling the final image. For larger images and higher iteration counts, the computation-to-communication ratio improves and efficiency would be expected to approach the theoretical maximum.

6.4 Improvements for a Production Version

- Replace static row assignment with a dynamic work queue (Task 5 optional) so faster workers receive more chunks automatically.
- Add worker heartbeats so the master detects failures faster than the full 15-second timeout.
- Use `epoll` instead of `select()` in the master for scalability to many workers.
- Add a reconnection mechanism allowing a restarted worker to resume from where it left off.
- Implement progress reporting so the user can see rendering completion percentage in real time.

7. Conclusions

This lab provided practical experience with all the core components of distributed systems programming in C: TCP socket programming, custom binary protocol design, master-worker architecture, fault tolerance using timeouts, and performance benchmarking. The Mandelbrot set was chosen as the computation task because its embarrassingly parallel nature makes it ideal for demonstrating the benefits of distributed computing.

Key lessons learned include: the importance of length-prefixed messages in stream-oriented TCP communication, the need for `read_all/write_all` wrappers to handle partial transfers, careful handling of floating-point endianness, and the difference between theoretical and practical speedup. The fault tolerance test clearly demonstrated how a production-grade master must never block indefinitely on a single worker.

The distributed system was tested end-to-end on a single machine using four concurrent processes simulating a real four-node cluster. All mandatory tasks were completed successfully and verified through terminal output, file comparisons, and visual image inspection.

8. References

7. CEF488 Laboratory Manual – Advanced C Programming for Networked Systems, Mr. Forcha Glen Beloa, Department of Network Engineering, 2025-2026.
8. W. Richard Stevens, Bill Fenner, Andrew M. Rudoff – Unix Network Programming, Volume 1: The Sockets Networking API, 3rd Edition, Addison-Wesley, 2003.
9. The Linux man-pages project – socket(7), tcp(7), select(2), read(2), write(2), htonl(3), htobe64(3). Available: <https://man7.org/linux/man-pages/>
10. ISO/IEC 9899:2011 – C11 Standard. Feature test macros: `_POSIX_C_SOURCE`, `_DEFAULT_SOURCE`.
11. IEEE 754-2008 – Standard for Floating-Point Arithmetic. Used as basis for double endianness handling via union reinterpretation.

Appendix A: Source Code Listings

A.1 mandelbrot.h

```
#ifndef MANDELBROT_H
#define MANDELBROT_H #include
<stdint.h>

void compute_row(unsigned char *pixels,
int row, int width, int height,
center_x, double center_y,
int max_iter);

void write_ppm(const char *filename,
const unsigned char *pixels,
int width, int height);

#endif /* MANDELBROT_H */
```

A.2 mandelbrot.c (key function)

```
void compute_row(unsigned char *pixels,
row, int width, int height,
double center_y,
double zoom, int max_iter)
{
    for (int col = 0; col < width; col++) {
        double
        cx = center_x + (col - width / 2.0) * zoom;
        double
        cy = center_y - (row - height / 2.0) * zoom;
        double
        zx = 0.0, zy = 0.0;
        int iter = 0;
        while
        (zx*zx + zy*zy <= 4.0 && iter < max_iter) {
            double tmp = zx*zx - zy*zy + cx;
            zy = 2.0 * zx *
            zy + cy;
            zx = tmp;
            iter++;
        }
        pixels[col] = (unsigned char) (255 * iter / max_iter);
    }
}
```

A.3 protocol.h – Message Opcodes and Structs

```
#define OPCODE_TASK 1
#define OPCODE_RESULT 3
#define OPCODE_TERMINATE 2

struct task_msg {
    uint32_t width, height, start_row,
    num_rows, max_iter;
    double center_x, center_y,
    zoom;
};

struct result_msg {
    uint32_t
    start_row, num_rows, width;
};

union double_bytes { double d; uint64_t u; };
```

```
int send_task(int sock, const struct task_msg *t); int
recv_task(int sock, struct task_msg *t); int send_result(int
sock, const struct result_msg *r, const
unsigned char *pixels); int recv_result(int sock, struct
result_msg *r, unsigned char **pixels_out);
int send_terminate(int sock);
int read_opcode(int sock);
```

A.4 Makefile

```
CC      = gcc
CFLAGS  = -Wall -Wextra -std=c11 -O2
LDFLAGS = -lm

all: local_test master worker perf_analysis

mandelbrot.o: mandelbrot.c mandelbrot.h
$(CC) $(CFLAGS) -c $< -o $@

protocol.o: protocol.c protocol.h
$(CC) $(CFLAGS) -c $< -o $@

local_test: local_test.o mandelbrot.o protocol.o
$(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

master: master.o mandelbrot.o protocol.o
$(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

worker: worker.o mandelbrot.o protocol.o
$(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

perf_analysis: perf_analysis.o mandelbrot.o protocol.o
$(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

clean: rm -f *.o local_test master worker
perf_analysis *.ppm
```